

String

string

string 是什么

`std::string` 是在标准库 `<string>`（注意不是 C 语言中的 `<string.h>` 库）中提供的一个类，本质上是 `std::basic_string<Tchar>` 的别称。

为什么要使用 string

在 C 语言中，提供了字符串的操作，但只能通过字符数组的方式来实现字符串。而 `string` 则是一个简单的类，使用简单，在 OI 竞赛中被广泛使用。并且相较于其他 STL 容器，`string` 的常数可以算是非常优秀的，基本与字符数组不相上下。

string 可以动态分配空间

和许多 STL 容器相同，`string` 能动态分配空间，这使得我们可以直接使用 `std::cin` 来输入，但其速度则同样较慢。这一点也同样让我们不必为内存而烦恼。

string 重载了加法运算符和比较运算符

`string` 的加法运算符可以直接拼接两个字符串或一个字符串和一个字符。和 `std::vector` 类似，`string` 重载了比较运算符，同样是按字典序比较的，所以我们可以直接调用 `std::sort` 对若干字符串进行排序。

使用方法

下面介绍 `string` 的基本操作，具体可看 [C++ 文档](#)。

声明

```
1 std::string s;
```

转 char 数组

在 C 语言里，也有很多字符串的函数，但是它们的参数都是 `char` 指针类型的，为了方便使用，`string` 有两个成员函数能够将自己转换为 `char` 指针——`data()`/`c_str()`（它们几乎是一样的，但最好使用 `c_str()`，因为 `c_str()` 保证末尾有空字符，而 `data()` 则不保证），如：

```
1 printf("%s", s);           // 编译错误
2 printf("%s", s.data());    // 编译通过，但是是 undefined behavior
3 printf("%s", s.c_str());   // 一定能够正确输出
```

获取长度

很多函数都可以返回 `string` 的长度：

```
1 printf("s 的长度为 %zu", s.size());
2 printf("s 的长度为 %zu", s.length());
3 printf("s 的长度为 %zu", strlen(s.c_str()));
```

▼ 这些函数的复杂度

`strlen()` 的复杂度一定是与字符串长度线性相关的。

`size()` 和 `length()` 的复杂度在 C++98 中没有指定，在 C++11 中被指定为常数复杂度。但在常见的编译器上，即便是 C++98，这两个函数的复杂度也是常数。

▼ Warning

这三个函数（以及下面将要提到的 `find` 函数）的返回值类型都是 `size_t`（`unsigned long`）。因此，这些返回值不支持直接与负数比较或运算，建议在需要时进行强制转换。

寻找某字符（串）第一次出现的位置

`find(str, pos)` 函数可以用来查找字符串中一个字符/字符串在 `pos`（含）之后第一次出现的位置（若不传参给 `pos` 则默认为 0）。如果没有出现，则返回 `string::npos`（被定义为 -1，但类型仍为 `size_t/unsigned long`）。

示例：

```
1 string s = "OI Wiki", t = "OI", u = "i";
2 int pos = 5;
3 printf("字符 I 在 s 的 %lu 位置第一次出现\n", s.find('I'));
4 printf("字符 a 在 s 的 %lu 位置第一次出现\n", s.find('a'));
5 printf("字符 a 在 s 的 %d 位置第一次出现\n", s.find('a'));
6 printf("字符串 t 在 s 的 %lu 位置第一次出现\n", s.find(t));
7 printf("在 s 中自 pos 位置起字符串 u 第一次出现在 %lu 位置", s.find(u, pos));
```

输出：

```
1 字符 I 在 s 的 1 位置第一次出现
2 字符 a 在 s 的 18446744073709551615 位置第一次出现 // 即为 size_t(-1)，具体数值与平台有关。
3 字符 a 在 s 的 -1 位置第一次出现 // 强制转换为 int 类型则正常输出 -1
4 字符串 t 在 s 的 0 位置第一次出现
5 在 s 中自 pos 位置起字符串 u 第一次出现在 6 位置
```

截取子串

`substr(pos, len)` 函数的参数返回从 `pos` 位置开始截取最多 `len` 个字符组成的字符串（如果从 `pos` 开始的后缀长度不足 `len` 则截取这个后缀）。

示例：

```
1 string s = "OI Wiki", t = "OI";
2 printf("从字符串 s 的第四位开始的最多三个字符构成的子串是 %s\n",
3       s.substr(3, 3).c_str());
4 printf("从字符串 t 的第二位开始的最多三个字符构成的子串是 %s",
5       t.substr(1, 3).c_str());
```

输出：

- 1 从字符串 `s` 的第四位开始的最多三个字符构成的子串是 `Wik`
- 2 从字符串 `t` 的第二位开始的最多三个字符构成的子串是 `I`

插入/删除字符（串）

`insert(index, count, ch)` 和 `insert(index, str)` 是比较常见的插入函数。它们分别表示在 `index` 处连续插入 `count` 次字符串 `ch` 和插入字符串 `str`。

`erase(index, count)` 函数将字符串 `index` 位置开始（含）的 `count` 个字符删除（若不传参给 `count` 则表示删去 `index` 位置及以后的所有字符）。

示例：

```
1 string s = "OI Wiki", t = " Wiki";
2 char u = '!';
3 s.erase(2);
4 printf("从字符串 s 的第三位开始删去所有字符后得到的字符串是 %s\n", s.c_str());
5 s.insert(2, t);
6 printf("在字符串 s 的第三位处插入字符串 t 后得到的字符串是 %s\n", s.c_str());
7 s.insert(7, 3, u);
8 printf("在字符串 s 的第八位处连续插入 3 次字符串 u 后得到的字符串是 %s",
9        s.c_str());
```

输出：

- 1 从字符串 `s` 的第三位开始删去所有字符后得到的字符串是 `OI`
- 2 在字符串 `s` 的第三位处插入字符串 `t` 后得到的字符串是 `OI Wiki`
- 3 在字符串 `s` 的第八位处连续插入 3 次字符串 `u` 后得到的字符串是 `OI Wiki!!!`

替换字符（串）

`replace(pos, count, str)` 和 `replace(first, last, str)` 是比较常见的替换函数。它们分别表示将从 `pos` 位置开始 `count` 个字符的子串替换为 `str` 以及将以 `first` 开始（含）、`last` 结束（不含）的子串替换为 `str`，其中 `first` 和 `last` 均为迭代器。

示例：

```
1 string s = "OI Wiki";
2 s.replace(2, 5, "");
3 printf("将字符串 s 的第 3~7 位替换为空串后得到的字符串是 %s\n", s.c_str());
4 s.replace(s.begin(), s.begin() + 2, "NOI");
5 printf("将字符串 s 的前两位替换为 NOI 后得到的字符串是 %s", s.c_str());
```

输出：

- 1 将字符串 `s` 的第 3~7 位替换为空串后得到的字符串是 `OI`
- 2 将字符串 `s` 的前两位替换为 `NOI` 后得到的字符串是 `NOI`

本页面最近更新：2024/11/18 17:31:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [lr1d](#), [Chrogeek](#), [DOHEX](#), [Enter-tainer](#), [hensier](#), [johnvp22](#), [MitsuPa](#), [onelittlechildawa](#),

[ouuan](#), [Tiphereth-A](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用